

数据结构与算法课程实习作业报告

孟语萱* 姜润孜 毛志远 史博文 梁亦奇

* 组长

摘要：本文围绕 AI Racer 赛道控制任务，设计并实现了一种基于单目左摄像头的视觉闭环控制算法。本算法将赛道路面抽象为图像中的深色连通区域，通过感兴趣区域截取、灰度阈值分割、形态学开闭运算和底部连通域筛选提取当前可行驶区域；随后利用带纵向权重的重心计算获得道路中心，并通过 PID 控制器产生转向量，不再以白线边界为主要依据。为提升复杂弯道中的稳定性，程序进一步引入基于上下半区道路中心差的内线偏置策略、转向低通滤波、丢失路面时保持历史误差的应急策略，以及依据曲率信号进行的分段限速和入弯保护。当前版本保留车辆检测、跟车与超车接口，但在稳定性优先原则下默认关闭多车交互逻辑。本文重点说明该算法的数据结构、核心函数、运行时间复杂度与适用范围。

关键字：图像阈值分割；连通域；重心计算；PID 控制；低通滤波；贪心策略；状态变量；单目视觉

目 录

1	算法思想	3
1.1	总体思路	3
1.2	算法流程图	4
1.3	算法运行时间分析	4
2	程序代码说明	5
2.1	数据结构说明	5
2.2	函数说明	6
2.3	程序限制	9
3	实验结果	9
3.1	测试数据	9
3.2	结果分析	10
4	实习过程总结	10
4.1	分工与合作	10
4.2	经验与教训	10
4.3	建议与设想	11
5	致谢	11

1 算法思想

1.1 总体思路

本项目的目标是在 Webots/AI Racer 仿真环境中，根据车辆左摄像头输入实时输出转向量与速度，从而使车辆尽可能稳定、快速地完成赛道行驶。由于控制接口在每一帧仅返回 `steering` 和 `speed` 两个连续量，本算法将问题分解为两个相互衔接的子问题：其一，从摄像头图像中估计当前道路中心及弯道方向；其二，根据道路中心误差与曲率变化生成转向和速度控制量。

从算法演进看，最初尝试直接沿用教学示例中的白线检测与边缘跟踪方案。该方案在 `basic` 赛道中能够运行，但在 `complex` 赛道的复合弯与急弯处，白线容易移出摄像头视野或严重偏离画面中心，导致控制器在关键时刻失去可靠参照。故单纯依赖白线的方案难以适应复杂赛道，因此后续实现将基于路面区域检测保障车辆正常运行。

当前版本首先在左摄像头图像中截取靠近车辆前方的感兴趣区域，将其灰度化后，进行固定阈值分割，将灰度较低的像素识别为候选路面。接下来通过形态学开运算去除小噪声、闭运算填补局部间隙，再使用轮廓查找获得若干连通域。为避免远处岔路或相邻路段干扰，程序优先选择与 ROI 底部接触的较大连通域，该连通域最可能对应车辆当前所在道路。

道路中心估计通过对连通域像素施加纵向权重后计算加权重心完成。靠近图像底部的区域在当前车辆控制中通常有更直接的约束意义，适当保留远处区域又有助于提前感知弯道，所以加权重心在二者之间折中。程序进一步比较 ROI 上半区与下半区的道路中心位置，估计前方道路相对于近处道路的横向偏移，并据此对中心点加入 APEX 内线偏置，使车辆在弯道中更早向内侧收线，以避免拐大弯耗费更多时间或面临更高撞墙风险。

转向控制采用比例-微分为主的 PID 形式。程序将道路中心横坐标与图像中心之差归一化为误差，利用比例项提供主要转向，利用微分项对误差变化作出提前响应，并将积分项置零以避免长时间偏差累积导致出弯过冲。PID 输出经限幅后进入一阶低通滤波器，从而减少瞬时图像噪声引起的方向抖动。若连续若干帧未检测到道路中心，控制器保留上一帧误差，并降低速度以避免弯道中短暂丢路时车辆直冲外墙。

速度控制采用与转向幅度和道路中心误差耦合的分段策略，本质上是一种基于局部状态的启发式贪心控制。程序将当前转向量和道路中心误差共同构造为曲率信号，根据曲率大小区分直道、轻弯、中弯、急弯和极急弯，并分别设定速度上限。当曲率信号单帧增幅超过阈值时，认为车辆正在从直道进入弯道，临时触发入弯限速，以防止直道高速延迟带入弯心造成外抛。曲率快速下降时，则允许提前出弯提速。

程序中保留了前车检测、跟车和超车相关接口，但该部分在当前版本默认关闭。第一阶段调试显示，车辆检测和横向超车偏移在复杂赛道中容易引入误判和额外不稳定性，在随后的多车比赛中亦发现对于车辆的检测反而降低了行进速度，使得性能下降。可以认为在检测车辆降低行驶风险和提升速度之间存在权衡取舍关系。

1.2 算法流程图

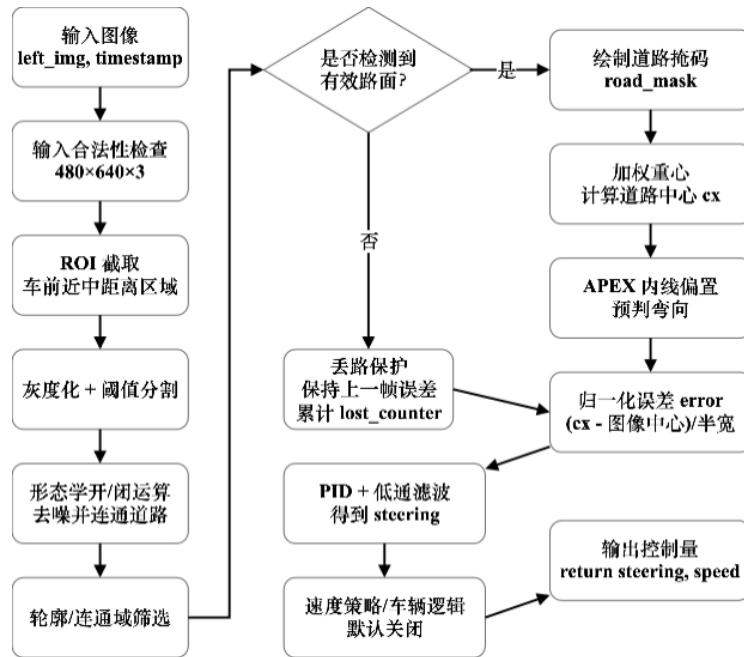


Figure 1 AI Racer 控制算法流程图

算法总体流程可以用如下文字流程图表示：

输入：左摄像头图像 `left_img` 与时间戳 `timestamp` → 检查图像是否为空且尺寸是否为 $480 \times 640 \times 3$ → 截取 ROI → 灰度化与阈值分割 → 形态学开闭运算 → 查找候选路面连通域 → 判断是否存在与 ROI 底部接触的有效连通域。

若存在有效连通域：选择面积最大的底部连通域 → 绘制道路掩码 → 计算纵向加权重心 → 比较上半区与下半区中心差 → 施加 APEX 内线偏置 → 得到道路中心 `cx`。

若不存在有效连通域：进入丢路保护 → 累加丢路帧数 → 保留上一帧误差方向 → 后续速度控制进入保护状态。

随后：计算归一化中心误差 `error` → 计算误差差分 `d_error` → PID 产生原始转向 `steer_raw` → 转向限幅与低通滤波 → 得到 `steering`。

速度控制阶段：由 `|steering|` 与 `|error|` 构造曲率信号 `s` → 根据 `s` 所在区间确定直道、轻弯、中弯、急弯或极急弯速度 → 根据曲率变化量 `ds` 进行入弯限速或出弯提速 → 在车辆逻辑关闭时直接输出最终 `steering` 与 `speed`。

SDK 输出：`control(left_img, right_img, timestamp)` 返回 `(steering, speed)`，供 Webots Driver 或平台控制接口调用。

1.3 算法运行时间分析

设输入图像尺寸为 $H \times W$ ，ROI 中像素数为 n 。图像灰度转换、固定阈值分割、形态学开闭运算、掩码遍历和加权重心计算均需对 ROI 像素进行常数扫描，因此时间复杂度为 $O(n)$ 。轮廓查找可视为对二值图像的一次线性遍历，其时间复杂度同样为 $O(n)$ ；候选轮廓筛选与面积比较的开销与轮廓点总数线性相关，而轮廓点总数不超过 ROI 像素数量级。

PID 计算、低通滤波、曲率信号更新、分段限速和输出限幅均只涉及常数个标量状态，时

间复杂度为 $O(1)$ 。由于当前版本默认关闭车辆检测与超车逻辑，主控制循环每帧的总体时间复杂度为 $O(n)$ ，空间复杂度也为 $O(n)$ ，主要用于二值图、道路掩码、索引网格和 OpenCV 轮廓结构。

在实际运行中， H 和 W 由平台固定为 480×640 ，因此从渐进意义看每帧计算时间可视为常数。主要时间开销集中在 OpenCV 图像处理环节，尤其是形态学运算、轮廓提取和掩码重心计算；相比之下，PID 控制和速度分段判断的开销可以忽略。若后续启用车辆检测、Canny 边缘检测和车辆候选轮廓筛选会额外引入一次 ROI 级线性扫描，但整体仍保持 $O(n)$ 量级。

各主要模块的渐进复杂度可归纳如下表所示，其中 n 表示道路检测 ROI 内像素数， n_v 表示可选车辆检测 ROI 内像素数。

Table 1 AI Racer 控制算法主要模块时间复杂度

模块	时间复杂度	主要作用
ROI 截取与灰度化	$O(n)$	生成用于阈值分割的灰度图
二值化与形态学处理	$O(n)$	获得较完整的道路候选掩码
轮廓/连通域筛选	$O(n)$	选择与车辆当前位置最相关的道路区域
加权重心与 APEX 偏置	$O(n)$	计算目标道路中心并预判弯道方向
PID、低通滤波与速度状态更新	$O(1)$	根据标量误差输出控制指令
车辆检测模块	$O(n_v)$	默认关闭；开启后额外执行一次 ROI 级图像处理

2 程序代码说明

2.1 数据结构说明

本程序没有额外定义复杂自定义类。输入图像 `left_img`、ROI、灰度图、二值图、道路掩码以及索引矩阵均以 `numpy.ndarray` 表示。该结构适合进行向量化像素运算，能够避免在 Python 层逐像素循环，从而降低解释器开销。

路面候选区域由 `cv2.findContours` 返回的轮廓列表表示。每个轮廓可以理解为隐式网格图中的一个连通分量的边界表示，程序通过面积、外接矩形底部位置和底部接触条件对连通域进行筛选。这一过程与课程中的图连通性思想相对应，即图像像素构成二维网格图，在阈值分割后，道路像素构成若干连通子图。随后，算法选择与车辆当前位置最相关的连通分量作为当前道路。

控制器状态主要由若干常数规模的全局变量维护，包括上一帧误差 `_pid_prev_error`、积分项 `_pid_integral`、低通滤波后的转向 `_steer_lpf`、丢路计数 `_lost_counter`、连续直道计数 `_straight_counter` 和上一帧曲率信号 `_prev_curve_signal`。这些参数负责在相邻帧之间维护并保存短期历史信息，使控制器在依赖当前图像的基础上对误差变化、曲率突变和检测失败作出平滑响应。

可选车辆逻辑中使用 `_vehicle_detected_queue` 维护最近若干帧是否检测到前车的历史并返回布尔类型，作用类似固定长度队列或滑动窗口。只有连续多帧均检测到车辆时，才确认前车存在，降低单帧误判对控制输出的影响。该逻辑在当前版本默认关闭，为多车交互提供接口。

Table 2 AI Racer 控制算法主要数据结构

数据结构	变量名
NumPy 数组	left_img、left_bgr、roi、gray
二值掩码矩阵	bin_img、road_mask、mask
列表	contours
顺序表	touching_candidates、fallback_candidates
固定长度队列	_vehicle_detected_queue

在此特别介绍一下 NumPy 数组类：ndarray 实现了多维数组数据结构，可用于更高效的矩阵运算等，在机器学习中被广泛应用。

```
class numpy.ndarray(shape, dtype=float, buffer=None,
                    offset=0, strides=None, order=None)
```

其中 shape 参数传入一个整数 tuple，用于描述实例化后的 ndarray 对象的形状。dtype 用于描述传入数据的类型。

ndarray 包括了一些属性。T 为 ndarray 转置后的数组视图、dtype 为数组元素的数据类型、flags 为 dict 类型存储的关于数组内存布局的信息、imag 为 ndarray 表示的数组的虚部、real 为 ndarray 表示的数组的实部、size 为数组中的元素总数、itemsize 为单个数组元素的大小（以字节为单位）、nbytes 为数组元素占用的总字节数、ndim 为数组的维数、shape 为表示数组维度的元组。此外还有一些其他属性。

ndarray 类上定义了许多方法。在数组（list）的基础上，ndarray 拓展了许多支持线性代数运算的方法，并且支持矩阵运算的许多操作。

2.2 函数说明

_clamp(v, lo, hi)：基础工具函数，将标量 v 限制在闭区间 $[lo, hi]$ 内。该函数用于转向量、速度、误差差分和车辆速度估计等多个位置，保证控制输出不会超出平台允许范围。该部分代码如下：

```
def _clamp(v: float, lo: float, hi: float) -> float:
    return float(max(lo, min(hi, v)))
```

detect_road_center(bgr)：路面检测与道路中心估计函数，输入一帧 BGR 图像，输出道路中心横坐标 cx；若未检测到有效路面则返回 None。函数内部依次完成 ROI 截取、灰度化、阈值分割、形态学去噪、轮廓提取、底部连通域筛选、纵向加权重心计算和 APEX 内线偏置，是整个视觉感知模块的核心。其代码较长，暂不展示。

detect_front_vehicle(bgr)：可选前车检测函数，返回是否检测到车辆、车辆中心横坐标和轮廓面积。该函数在图像中部区域使用灰度边缘和轮廓面积约束寻找前车候选，并通过底部位置约束排除远处或背景干扰。当前版本中由于 ENABLE_VEHICLE_LOGIC=False，该函数不会参与主控制输出。代码如下所示：

```

def detect_front_vehicle(bgr: np.ndarray) -> Tuple[bool, float, float]:
    h, w = bgr.shape[:2]
    roi_top = int(h * 0.4)
    roi_bottom = int(h * 0.9)
    roi_left = int(w * 0.25)
    roi_right = int(w * 0.75)
    roi = bgr[roi_top:roi_bottom, roi_left:roi_right]
    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 50, 150)
    kernel = np.ones((5, 5), np.uint8)
    edges = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel)
    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)

    best_area = 0.0
    best_cx = 0.0
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area < VEHICLE_MIN_AREA:
            continue
        x, y, wb, hb = cv2.boundingRect(cnt)
        bottom_y = y + hb
        dist_to_bottom = (roi_bottom - roi_top) - bottom_y
        if dist_to_bottom < VEHICLE_BOTTOM_MARGIN and area > best_area:
            M = cv2.moments(cnt)
            if M["m00"] != 0:
                best_cx = M["m10"] / M["m00"] + roi_left
                best_area = area
    if best_area > 0:
        return True, float(best_cx), float(best_area)
    return False, 0.0, 0.0

```

update_vehicle_speed(area, dt=0.032): 可选车辆速度估计函数，根据前车轮廓面积及其相对变化率估计前车速度，并使用指数平滑更新 _vehicle_speed_est。该函数用于跟车逻辑，以避免因前车距离变化导致速度输出剧烈震荡。代码如下所示：

```

def update_vehicle_speed(area: float, dt: float = 0.032) -> float:
    global _prev_area, _vehicle_speed_est
    if _prev_area <= 0:
        _prev_area = area
        return 0.7
    area_rate = (area - _prev_area) / max(1e-6, _prev_area)
    area_speed = 1.0 - (area - 1500) / 6000.0
    area_speed = _clamp(area_speed, 0.45, 1.0)
    speed = area_speed * (1.0 - 0.35 * max(0.0, area_rate))
    speed = _clamp(speed, 0.35, 1.0)
    _vehicle_speed_est = 0.7 * _vehicle_speed_est + 0.3 * speed

```



```

_prev_area = area
return _vehicle_speed_est

```

`compute_steering(left_bgr)`: 转向计算函数。该函数调用 `detect_road_center` 获得道路中心, 进而计算归一化误差。当道路中心缺失时, 函数进入丢路保护并保留上一帧误差。随后函数依据比例项、积分项和微分项计算 `steer_raw`, 并经过限幅与低通滤波得到最终 `steering`。函数代码如下所示:

```

def compute_steering(left_bgr: np.ndarray) -> float:
    global _pid_prev_error, _pid_integral, _steer_lpf, _last_steering, _lost_counter
    cx = detect_road_center(left_bgr)
    if cx is not None:
        _lost_counter = 0
        error = (cx - IMG_W / 2.0) / (IMG_W / 2.0)
    else:
        _lost_counter += 1
        # 关键修改: 弯中丢路时不要快速把方向衰减到 0, 否则会直冲外墙。0
        error = _pid_prev_error * LOST_ERROR_KEEP
    dt = 0.032
    d_error = _clamp((error - _pid_prev_error) / dt, -8.0, 8.0)
    _pid_integral = _clamp(_pid_integral + error * dt, -1.0, 1.0)
    steer_raw = KP * error + KI * _pid_integral + KD * d_error
    steer_raw = _clamp(steer_raw, -MAX_STEER, MAX_STEER)
    _pid_prev_error = error
    _steer_lpf = STEER_ALPHA * steer_raw + (1.0 - STEER_ALPHA) * _steer_lpf
    steering = _clamp(_steer_lpf, -MAX_STEER, MAX_STEER)
    _last_steering = steering
    return steering

```

`_speed_from_steering(steering)`: 分段限速函数。该函数综合当前转向幅度与道路中心误差构造曲率信号, 并依据曲率大小选择直道、轻弯、中弯、急弯或极急弯速度。函数根据曲率上升判断是否处于入弯瞬间从而触发临时限速, 或根据曲率下降判断是否可以提前出弯加速。代码较长, 暂不展示。

`compute_speed_and_overtake(steering, left_bgr, timestamp)`: 速度与可选超车逻辑函数。当前版本在车辆逻辑关闭时直接返回基础速度与转向输出。开启车辆逻辑时, 该函数会整合前车检测、滑动窗口确认、跟车速度估计和短时超车方向偏置。代码较长, 暂不展示。

`control(left_img, right_img, timestamp)`: 平台 SDK 入口函数。该函数首先检查输入图像合法性, 然后调用 `compute_steering` 与 `compute_speed_and_overtake` 生成控制量。若运行过程中出现异常, 则返回安全低速与零转向, 避免程序崩溃直接中断比赛。

```

def control(
    left_img: np.ndarray,
    right_img: np.ndarray,
    timestamp: float,
) -> tuple[float, float]:
    if left_img is None or left_img.size == 0 or left_img.shape != (IMG_H, IMG_W, 3):
        return 0.0, 0.0

```



```
try:
    steering = compute_steering(left_img)
    speed, steering_out = compute_speed_and_overtake(steering, left_img, timestamp)
except Exception:
    steering_out, speed = 0.0, 0.1
return float(steering_out), float(speed)
```

`_get_device(robot, names)` 与 `run()`：本地 Webots 主循环辅助函数。前者按照候选名称寻找设备，后者初始化 Driver 或 Robot API、启用左摄像头并在仿真循环中持续读取图像、调用控制器、设置巡航速度和转向角。该部分主要用于本地调试，与在线平台的 `control` 接口保持兼容。

2.3 程序限制

算法依赖赛道与背景之间存在较稳定的灰度差异。若光照、阴影、材质或渲染设置发生明显变化，固定 `DARK_THRESHOLD` 可能无法稳定区分深色路面与背景，进而造成道路区域漏检或误检。

算法默认输入图像尺寸为 $480 \times 640 \times 3$ 。`control` 函数会对图像尺寸进行严格检查，若平台输入尺寸或通道格式发生变化，函数将直接返回零速度与零转向，需注意。

底部连通域筛选虽然能够降低相邻路段和远处岔路的干扰，但在极端弯道、道路被遮挡或多个深色区域同时接触 ROI 底部时，仍可能选择错误连通域。此时加权重心会偏离真实道路中心，导致转向输出异常。

APEX 内线偏置属于启发式策略，其效果取决于赛道宽度、弯道半径和车辆动力学。偏置过小会削弱入弯提前量，偏置过大则可能导致车辆压向内侧边界甚至冲出路面。

程序使用全局变量保存控制器历史状态，便于在连续帧之间保持记忆，但这也意味着控制器不是无状态函数，为了维护控制器的历史状态，需要进行额外的开销，包括存储上的开销和时间上的开销，后者更可能对运行速度造成一定影响。

3 实验结果

3.1 测试数据

绝大多数测试均在在线平台完成，在单车测试复杂赛道时发现车辆检测到转弯降速过于严重。此问题修改后，在多车测试环节，车辆能够正常行驶且不与其他车辆发生碰撞。

平台上对战测试的结果如下表所示；可以看到算法的表现比较稳定，单圈用时在不同赛事中，随着车辆数目的变化和行车距离的改变均没有明显变化。同时与其他组相比，我们的策略使虚拟小车在速度方面体现出更优异的性能。

Table 3 测试/比赛结果 (name=kilo)

测试/比赛结果	单圈用时	总用时	名次
排位赛-第 1 场	216 s	216 s	1
小组赛-第 2 场	210.6 s	632.7 s	1
半决赛-第 2 场	210.6 s	632.7 s	1
决赛	210.7 s	1053.2 s	1

3.2 结果分析

首先，在本地测试中发现，原先基于白线检测的算法由于道路中央白线不连续，会发生抖动，并且一旦白线脱离 camera 视野范围，车辆就无法重新回到道路中行驶。因此修改控制器的逻辑，用 camera 获取路面信息，并基于路面信息进行判断，避免了白线丢失造成车辆失控的问题。

在平台上单车测试发现，直线速度对于车辆速度影响十分明显，为加快车速，修改控制器参数使其在直线情况下（一般状态）具有最大的速度。单圈速度随后提升。车辆在进行转弯时，基础的转弯逻辑会导致速度骤降，单纯减少转弯减速又容易致转弯失败，故通过道路检测判断曲率，将转弯划归不同类别，设置不同的转弯方案，提升了速度，解决了这一问题。

在平台对战测试中可以发现，更精细的车辆检测/超车逻辑/跟车逻辑反而对于速度产生不利的影响。不过算法运行时间整体在预期之中。主要的时间开销发生在检测弯道、检测车辆等环节。

4 实习过程总结

4.1 分工与合作

- 孟语萱为组长，负责对接开发组以及代码的初步修改与策略设计；
- 姜润孜、毛志远负责为控制器增加动态车辆检测等功能，设计并改进算法策略；
- 史博文负责参数调试与性能优化的工作；
- 梁亦奇负责对代码进行初步的修改和调参，并对接至在线平台运行。

4.2 经验与教训

最开始在原始版本单车测试时，发现速度过于缓慢，尝试进行提速，将直线行驶速度设置为最大值，起到改善作用。在过程中，每次参数调试均只改变几个参数从而明确参数对于控制器性能的影响。

虽然进行精心的算法策略设计和参数调试，但是在最后赛事时关于车辆检测和超车的逻辑处于默认关闭状态，反而达到了更优的性能。或许实际情况下精细的策略设计不一定能像预期中一般改善算法表现。

4.3 建议与设想

从本次实习的技术过程看, AI Racer 任务能够较好地把数据结构、图像处理、状态维护和控制策略结合起来。后续继续完善该项目建议首先建立统一的测试与日志系统, 在每次提交时记录赛道、参数、圈数、是否碰撞、平均速度、丢路帧数和最高曲率等指标, 以避免单纯凭主观印象调参。

在算法层面可以尝试引入更自适应的路面分割方法, 例如基于 ROI 内灰度分布动态选择阈值, 或在 HSV/灰度空间中综合颜色与亮度信息, 以减少固定阈值对光照条件的依赖。

对于道路中心估计, 可以进一步加入透视变换、滑动窗口或纯跟踪点算法, 使控制目标从简单重心扩展为一段可预测的局部轨迹。

在控制层面, 当前分段限速与 PID 参数均为经验策略, 后续可设计离线参数搜索或网格化测试脚本, 对 KP、KD、STEER_ALPHA、APEX 增益和各段速度阈值进行系统优化。

若需要进一步提高性能, 也可在保持实时性的前提下尝试 Pure Pursuit、Stanley 控制或简化模型预测控制等路径跟踪方法。

5 致谢

感谢陈斌老师, 感谢助教老师, 感谢小组成员, 感谢 AI_Racer 技术组。

6 参考文献

- [1] OpenCV Documentation. Image Thresholding. https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html.
- [2] OpenCV Documentation. Morphological Transformations. https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html.
- [3] OpenCV Documentation. Contours: Getting Started. https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html.
- [4] OpenCV Documentation. Canny Edge Detection. https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html.
- [5] Canny, J. A computational approach to edge detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986, PAMI-8(6): 679–698.
- [6] Ziegler, J. G., Nichols, N. B. Optimum settings for automatic controllers. Transactions of the ASME, 1942, 64: 759–765.
- [7] NumPy Developers. numpy.ndarray. <https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html>.
- [8] Cyberbotics Ltd. Webots Automobile Driver Library. <https://cyberbotics.com/doc/automobile/driver-library>.
- [9] Cyberbotics Ltd. C++, Java and Python Wrappers of the Automobile Libraries. <https://cyberbotics.com/doc/automobile/cpp-java-and-python-wrappers-of-the-automobile-libraries>.